

INTERVIEW DOCS

Spring & Spring Boot Interview Questions

Frequently Asked Spring Ecosystem Questions with Detailed Answers

Java Agentic AI — Knowledge Base Reference

1. Core Spring

Q1. What is Dependency Injection and why does it matter?

DI is a technique where an object's dependencies are supplied from outside rather than created internally with 'new'. It decouples classes from the concrete implementations they depend on, making code easier to unit test (dependencies can be mocked), more flexible (swap implementations without changing consumer code), and centrally configurable via the Spring IoC container.

Q2. Constructor vs field vs setter injection — which is best and why?

Constructor injection is generally preferred: it allows dependencies to be declared final (immutable), makes required dependencies explicit and impossible to forget, and lets the class be instantiated and tested with plain 'new' outside of Spring. Field injection (`@Autowired` directly on a field) is discouraged because it hides dependencies, prevents immutability, and makes unit testing without reflection-based tools harder.

Q3. What is a Spring bean's default scope, and what other scopes exist?

The default scope is singleton — exactly one shared instance per Spring container. Other scopes include prototype (new instance per request for the bean), and the web-aware request/session scopes (one instance per HTTP request/session).

Q4. What's the difference between `@Component`, `@Service`, and `@Repository`?

All three are stereotype annotations that register a class as a Spring-managed bean discovered via component scanning; functionally, `@Service` and `@Repository` are specializations of `@Component` used to signal architectural intent (business logic vs data access layer). `@Repository` additionally enables automatic translation of persistence-specific exceptions into Spring's unified `DataAccessException` hierarchy.

2. Spring Boot

Q5. How does Spring Boot's auto-configuration actually work?

Auto-configuration classes are conditionally registered using `@Conditional` annotations (like `@ConditionalOnClass` and `@ConditionalOnMissingBean`). At startup, Spring Boot inspects what's on the classpath and what beans already exist, then activates only the configuration that makes sense — for example, seeing an H2 driver on the classpath but no user-defined `DataSource` bean triggers an auto-configured in-memory `DataSource`.

Q6. What does `@SpringBootApplication` actually consist of?

It's a meta-annotation combining `@Configuration` (marks the class as a bean definition source), `@EnableAutoConfiguration` (triggers the auto-configuration engine), and `@ComponentScan` (scans the current package and sub-packages for stereotype-annotated beans).

Q7. How would you externalize configuration for different environments?

Using Spring Profiles: define application-dev.yml, application-prod.yml, etc., and activate the desired one via spring.profiles.active (as an environment variable, JVM arg, or in the base application.yml). Profile-specific beans can also be conditionally registered with @Profile("dev").

3. Spring MVC / REST

Q8. Explain the request lifecycle through Spring MVC.

A request first hits the DispatcherServlet (the front controller). HandlerMapping resolves which controller method matches the URL/method/headers. HandlerAdapter invokes that method. The controller runs business logic and returns either a view name + model (traditional MVC) or a response body directly (REST, via @ResponseBody / @RestController). For traditional MVC, a ViewResolver then maps the logical view name to an actual template, which renders the final response.

Q9. @Controller vs @RestController?

@Controller methods return a logical view name to be resolved into an HTML template unless explicitly annotated with @ResponseBody. @RestController is a shorthand combining @Controller and @ResponseBody at the class level, so every method's return value is serialized directly into the response body (typically JSON) — the standard choice for REST APIs.

Q10. How do you handle exceptions globally in a Spring REST API?

With a class annotated @RestControllerAdvice containing @ExceptionHandler methods for specific exception types (and often a catch-all for generic Exception). This centralizes error-to-response mapping across every controller instead of repeating try-catch blocks everywhere.

4. Spring Data JPA

Q11. What is the N+1 select problem and how do you solve it?

Fetching N parent entities with a lazily-loaded association triggers 1 query for the parents plus N additional queries — one per parent — when each association is accessed, instead of one efficient join. Fix it with JOIN FETCH in a JPQL query, or the @EntityGraph annotation, to load the association in the same query.

Q12. Why does calling a @Transactional method from within the same class sometimes not work?

Spring's declarative transaction management relies on an AOP proxy that wraps the bean; the transactional advice only runs when the method is invoked through that proxy (i.e. from outside the class, or from another Spring-managed bean). Calling the method directly via 'this' inside the same class bypasses the proxy entirely, so no transaction boundary is applied — a classic self-invocation pitfall.

Q13. What's the difference between JPA, Hibernate, and Spring Data JPA?

JPA is a specification describing how Java objects map to relational tables. Hibernate is the most widely used concrete implementation of that specification. Spring Data JPA sits on top of JPA and

auto-generates repository implementations from interface method names and annotations, eliminating manual DAO boilerplate.

5. Spring Security

Q14. Authentication vs authorization in Spring Security?

Authentication establishes who the user is (verifying credentials, tokens); authorization determines what that authenticated user is permitted to do, typically expressed as roles or granted authorities checked via `authorizeHttpRequests` rules or method-level `@PreAuthorize` annotations.

Q15. Why is BCrypt preferred over MD5/SHA for storing passwords?

BCrypt is deliberately slow and computationally expensive, and it automatically incorporates a random salt per password — both properties make brute-force and rainbow-table attacks far less practical. MD5/SHA are fast, general-purpose hash functions designed for integrity checks, not password storage, and are considered unsafe for that purpose today.

Q16. How would you secure a stateless REST API?

Configure `SessionCreationPolicy.STATELESS` (no server-side session), issue signed JWTs on login, and add a custom filter that validates the token's signature/expiry on each request and populates the `SecurityContext` manually. CSRF protection is typically disabled for such APIs since there's no ambient browser session to forge in a token-authenticated architecture.

6. Microservices

Q17. What is a circuit breaker and why is it needed?

A circuit breaker (e.g. Resilience4j) monitors calls to a downstream dependency; once failures exceed a threshold, it 'opens' and fails fast (or invokes a fallback) instead of continuing to call a service that's likely to keep failing — this prevents cascading failures and thread exhaustion across the whole system while giving the failing dependency time to recover.

Q18. Why does each microservice typically own its own database?

Sharing a database across services creates tight coupling — any schema change risks breaking unrelated services and undermines independent deployability. Database-per-service enforces proper service boundaries, at the cost of needing patterns like the Saga pattern to maintain consistency across services for operations that span multiple of them.

Q19. How does service discovery work (e.g. with Eureka)?

Each service instance registers itself with a discovery server on startup and sends periodic heartbeats. Client services query the registry by logical service name (rather than a hardcoded host/port) to find healthy instances, with client-side load balancing distributing calls across them.

7. Rapid-Fire Recap Table

Question	One-line Answer
What replaced WebSecurityConfigurerAdapter?	A SecurityFilterChain @Bean using the fluent HttpSecurity DSL
What annotation enables method-level security?	@EnableMethodSecurity, then @PreAuthorize/@PostAuthorize on methods
Default port for a Spring Boot web app?	8080
What does @Valid trigger?	Bean Validation (Hibernate Validator) on the annotated request body
What replaced Hystrix?	Resilience4j
What is the purpose of an API Gateway?	Single entry point routing requests and centralizing cross-cutting concerns